Rotman

INTRO TO SQL

Workshop Site: https://rmdal.github.io/mfrm-sql-2026/

November 3, 2026 Prepared by Jay / MDAL



Rotman School of Management
UNIVERSITY OF TORONTO

What's SQL (Structured Query Language)

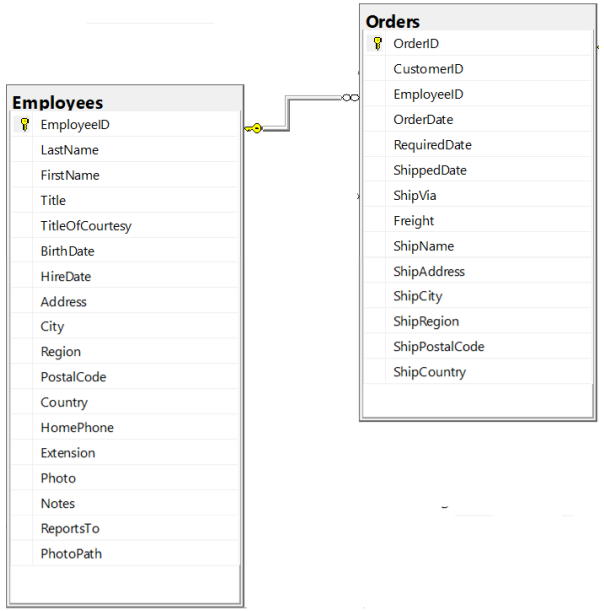
- Most widely used database (DB) language
  - a domain specific language: managing data stored in *relational DBs*
- Not a proprietary language
  - Open specifications/standards (ANSI & ISO)
  - All major *DBMS* (DB Mgmt. System) vendors implement Standard SQL
  - However, SQL Extensions are usually vendor specific (SQL dialects)
    - Every major DBMS vendor modifies the SQL to fit its own architecture, performance goals, and extra capabilities
- Powerful despite simplicity

ANSI - American National Standards Institute; ISO – International Organization for Standardization

What's a DB and a Relational DB

- What's a database: A collection of data in an organized way

- *Relational DB (RDB)*
  - tables
    - columns/fields/variables, and a datatype per column
    - rows/records/observations
  - primary key, foreign key, constraints, and relationships between tables
  - other objects: indices, views, triggers and many more



What's a DB Management System

- A software system that manages/maintains DBs

- A few examples of Relational DBMS (RDBMS)
  - Open source: SQLite, DuckDB, MariaDB, PostgreSQL
  - Commercial: MySQL, Microsoft SQL Server, Oracle, etc.

Our choice for this workshop







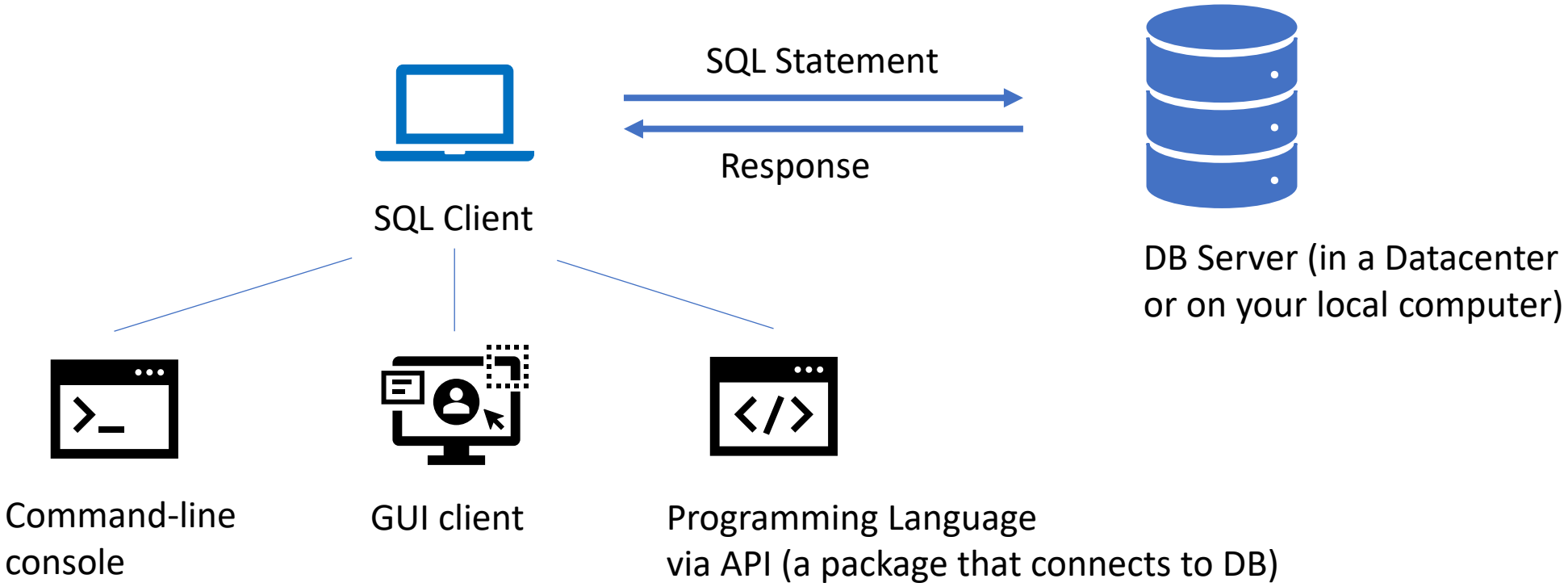






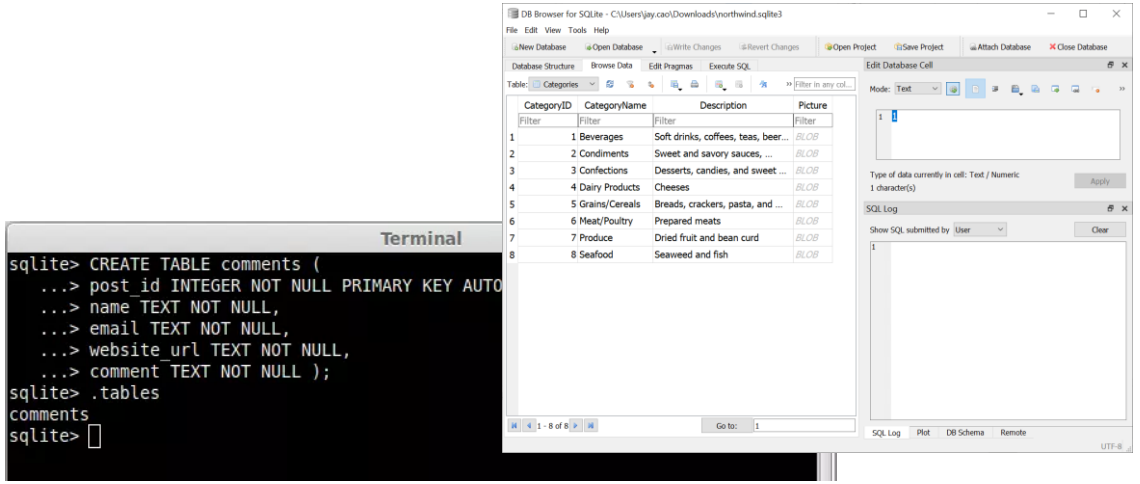


How to use SQL – A Typical Setup



Note: Our DB choice for this workshop, SQLite, isn't a client-server DB engine. It's an embedded (or "server-less") DB. It runs directly inside an application process and stores data in a single file on local disk.

SQL Clients



- DB specific management client
  - Command-line console
  - GUI (Graphic User Interface) client
    - e.g., DB Browser for SQLite, MySQL Workbench, pgAdmin for PostgreSQL, MS SSMS

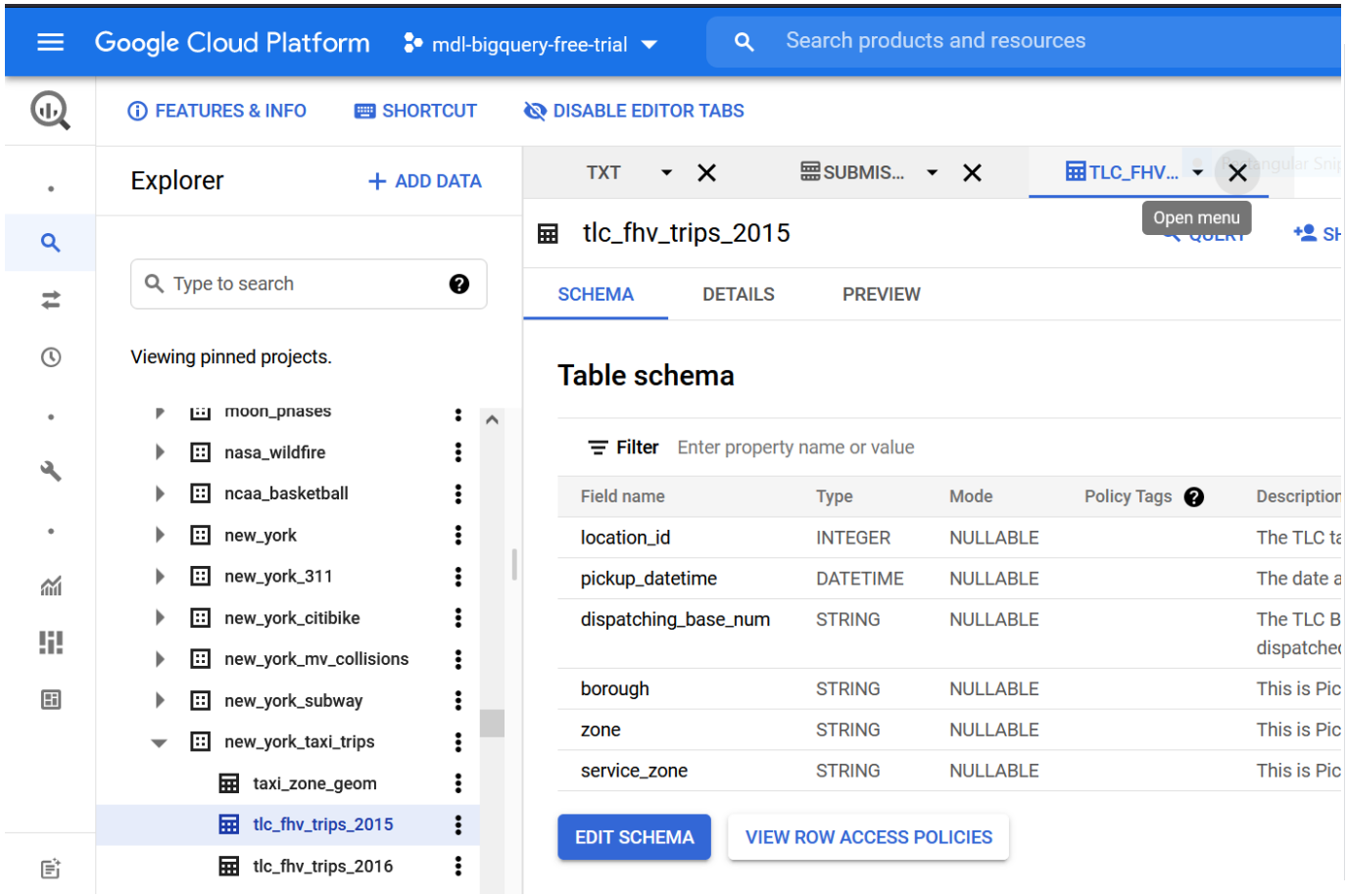
- Generic DB client can connect to different DBs through connectors
  - GUI client (e.g. DBeaver, Beekeeper Studio, Navicat)
  - Programming language
    - e.g., Python + SQLAlchemy + DBAPI (e.g. connect to SQLite, MySQL, etc.), R + dbplyr
    - **In this workshop**: Python notebook with ipython-sql (with SQLAlchemy as a its dependency) + SQLite (ships with Python)

Beyond a relational DB language

- SAS's PROC SQL
- Spark's SparkSQL
  - Apache Spark is a big data computing framework
- Hive's HiveQL, an SQL-like query language
  - Apache Hive is a distributed data warehouse (data warehouse?)
- Google BigQuery's SQL
  - BigQuery is Google's data warehouse (analyze petabytes of data at ease)

ref. 1) DB vs data warehouse vs data lake; 3) NoSQL DB vs SQL DB

Big Data **ML** with SQL (e.g. Google BQML)




```
CREATE OR REPLACE MODEL taxi.taxifare_model_2
OPTIONS
  (model_type='linear_reg', labels=['total_fare']) AS
WITH params AS (
    SELECT
    1 AS TRAIN,
    2 AS EVAL
    ),
  daynames AS
    (SELECT ['Sun', 'Mon', 'Tues', 'Wed', 'Thurs', 'Fri',
'Sat'] AS daysofweek),
  taxitrips AS (
  SELECT
    (tolls_amount + fare_amount) AS total_fare,
    daysofweek[ORDINAL(EXTRACT(DAYOFWEEK FROM
pickup_datetime))] AS dayofweek,
    EXTRACT(HOUR FROM pickup_datetime) AS hourofday,
    SQRT(POW((pickup_longitude - dropoff_longitude),2) +
POW(( pickup_latitude - dropoff_latitude), 2)) as dist,
#Euclidean distance between pickup and drop off
    SQRT(POW((pickup_longitude - dropoff_longitude),2))
as longitude, #Euclidean distance between pickup and drop
off in longitude
    SQRT(POW((pickup_latitude - dropoff_latitude), 2)) as
...
```

Ref. 1) [Using BigQuery ML and BigQuery GIS together to predict NYC taxi trip cost | Google Cloud Blog]

2) [Predict Taxi Fare with a BigQuery ML Forecasting Model | Google Skills]

Go Big with SQL/RDB Even on a Local Machine

- Todd W. Schneider's original analysis on NYC Taxi Trips (2015)
  - 267 GB of raw data
  - 1.1 billion rows
  - **PostgreSQL** and R
  - Macbook Pro then (2015), but now Macbook Air should be able to handle

- NYC Taxi Trips data with DuckDB
  - 1.1 Billion Taxi Rides using DuckDB
  - Driving CSV Performance: Benchmarking DuckDB with the NYC Taxi Dataset

Ref. Analyzing 1.1 Billion NYC Taxi and Uber Trips, with a Vengeance - Todd W. Schneider (toddwschneider.com)

Why Learn SQL

- Universal language to access, manage, and analyze data
  - Most companies runs some RDBs and in addition SQL goes beyond RDBs
- Allow you to handle large structured datasets
  - DBMS has specialized memory management system so you can even process large data on a local PC with SQL
- SQL skill is almost always required for data analytics related jobs
  - i.e., SQL questions show up in interviews
  - AI can write SQL better, so why: understand, verify, make decision…

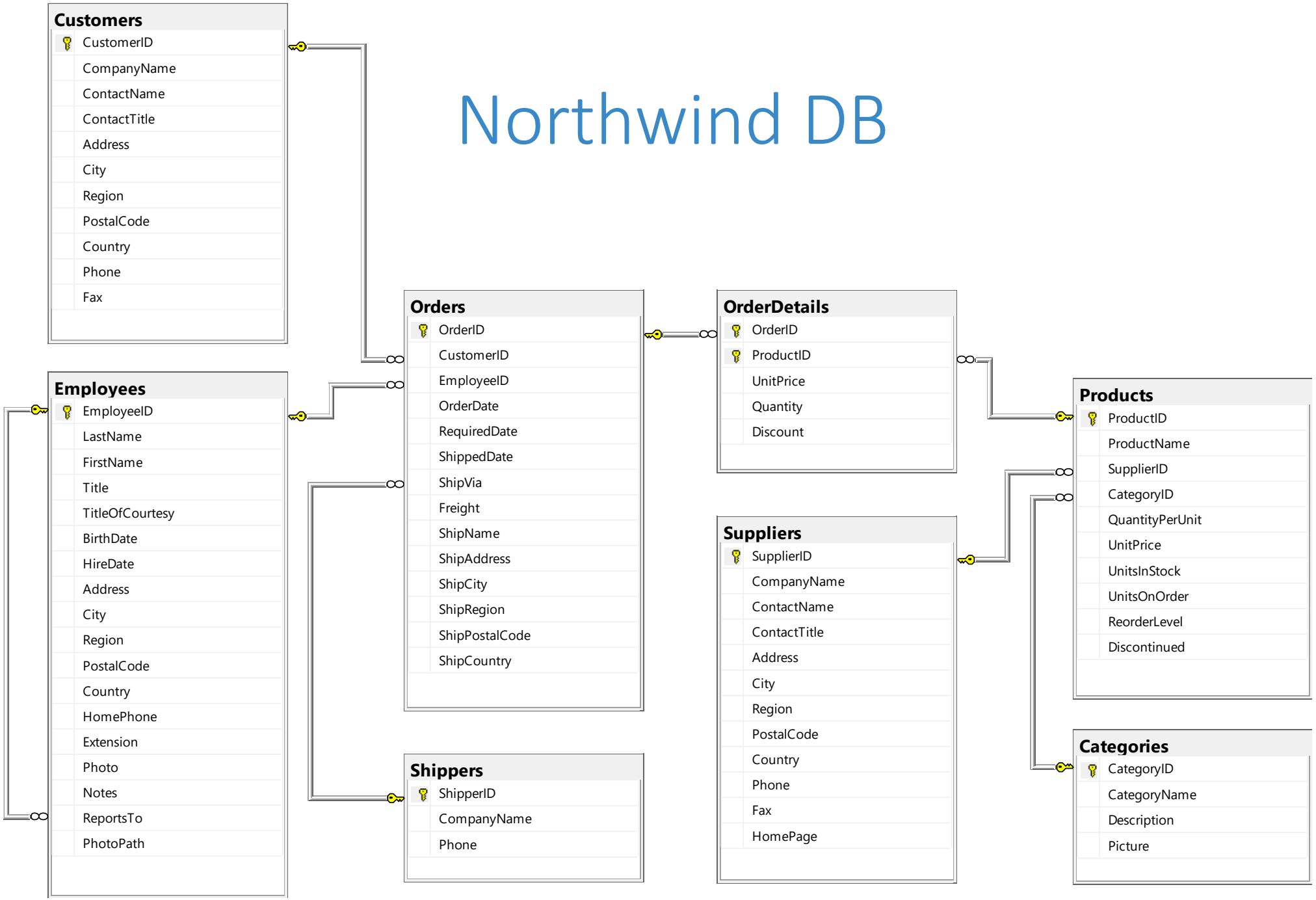
Hands-on SQL Learning

- Course website: https://rmdal.github.io/mfrm-sql-2026/

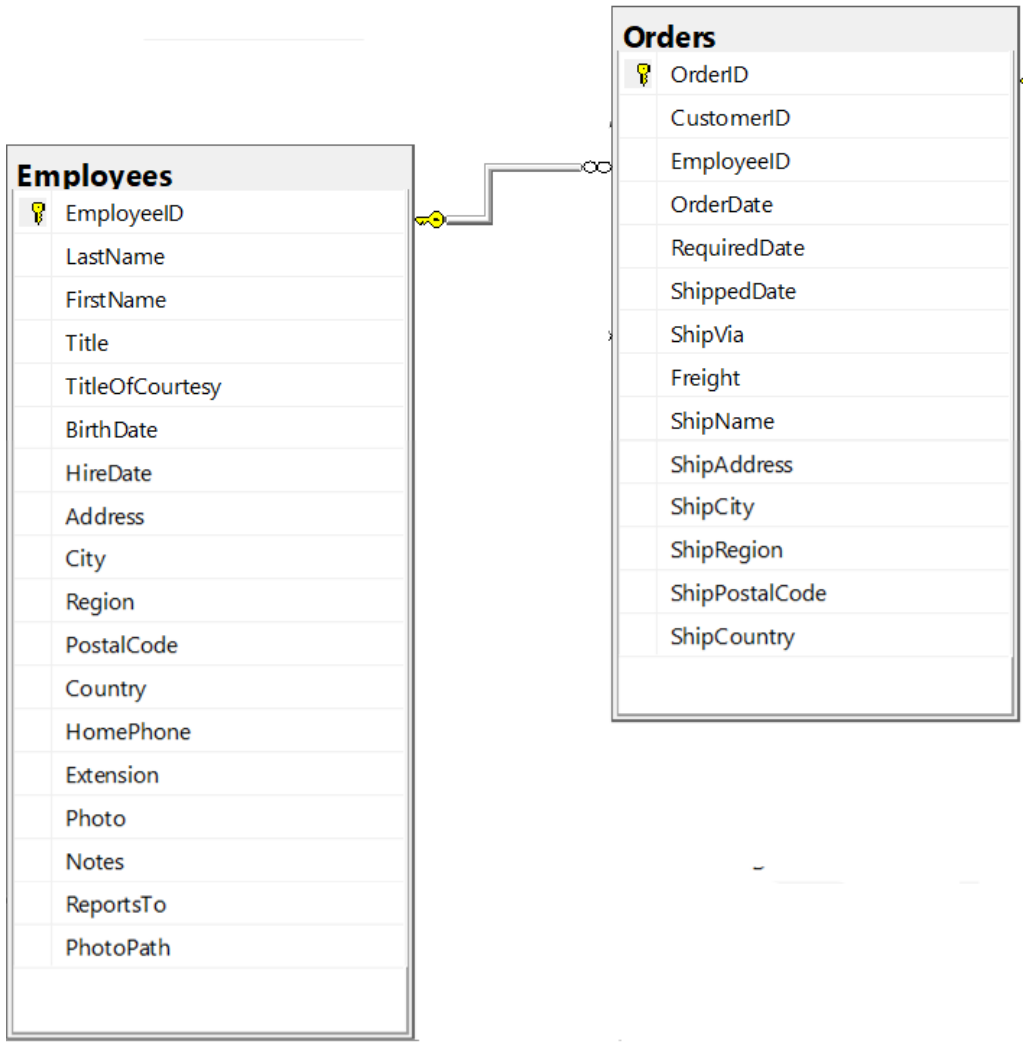
- Google Colab
  - Google's Jupyter Notebook
  - A notebook can contain live code, equations, visualizations and narrative text

- Why SQLite?
  - a small, fast, self-contained, high-reliability, full-featured, SQL DB engine
  - perfect for learning SQL

Northwind DB



Primary key (PK), foreign key (FK), constraints & relationships - 1



EmployeeID	LastName	FirstName	Title	…
1	Davolio	Nancy	Sales Representative	…
2	Fuller	Andrew	Vice President, Sales	…
3	Leverling	Janet	Sales Representative	…
4	Peacock	Margaret	Sales Representative	…
…	…	…	…	…

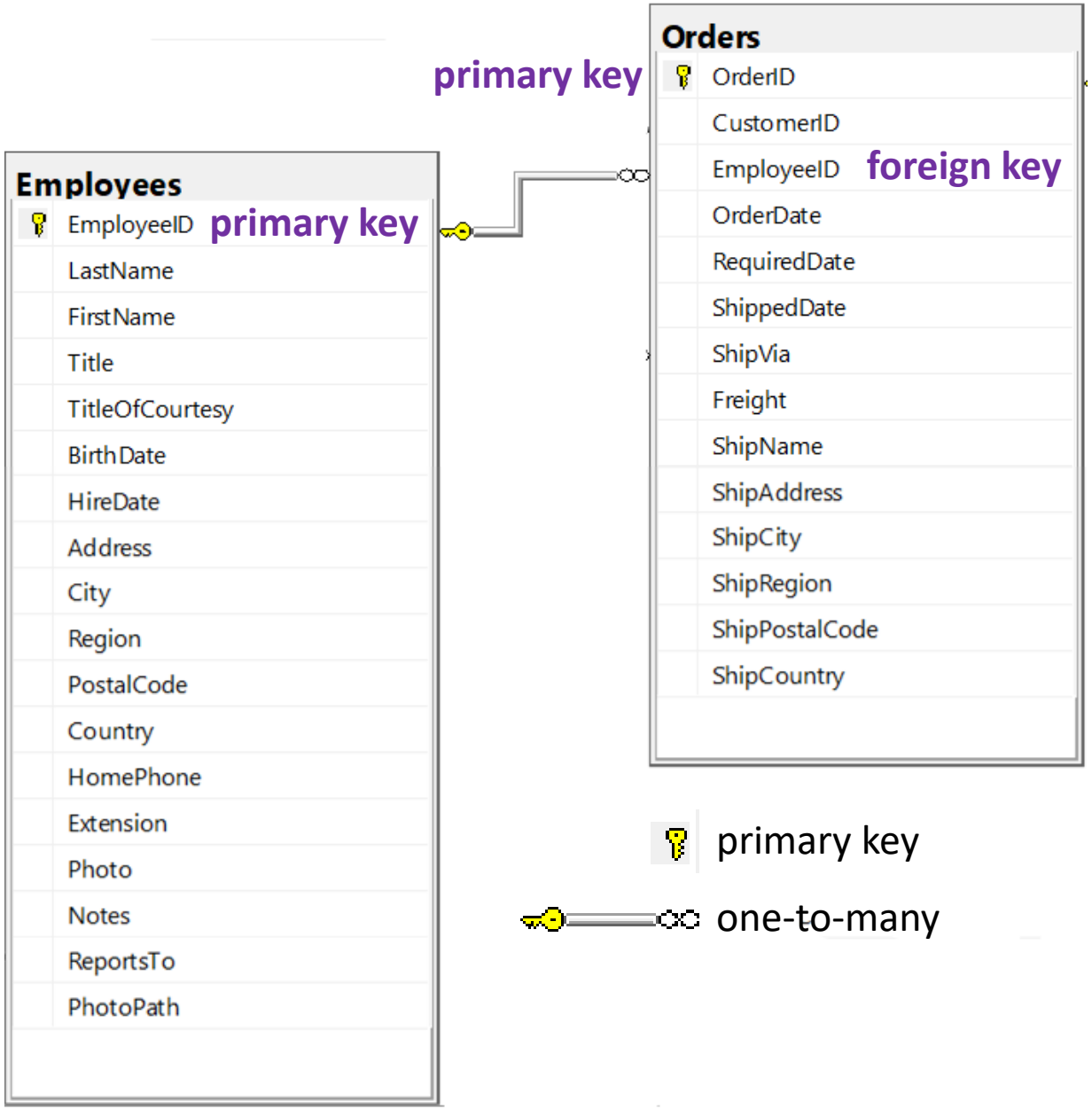
OrderID	CustomerID	EmployeeID	…
10248	VINET	5	…
10249	TOMSP	6	…
10250	HANAR	4	…
…	…	…	…

PK, FK, constraints & relationships - 2

- Two keys
  - **primary key**: uniquely identifies an observation in its own table
  - **foreign key**: uniquely identifies an observation in another table (parent table), or sometimes in the same table

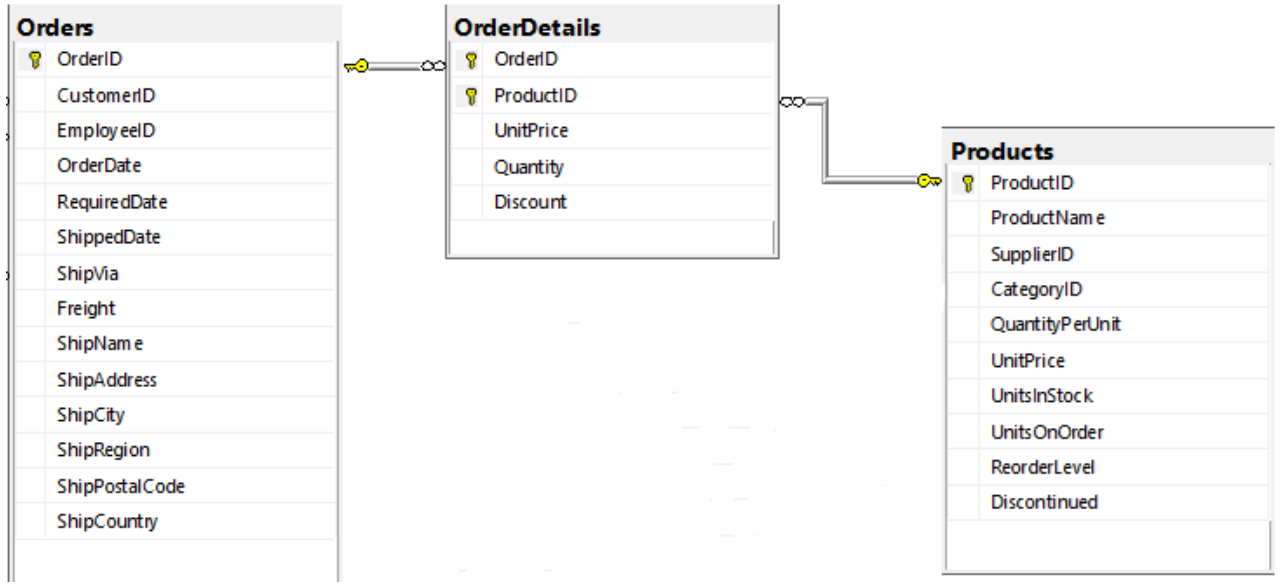
- Relationship between tables
  - one-to-one
  - **one-to-many**
  - many-to-many*

- FK constraints maintain *referential integrity*
  - Prevent "orphan records“, situations where a child record references a parent record that no longer exists



PK, FK, constraints & relationships - 3

- Many-to-many relationship
  - Common in conceptual business logic (e.g., students & courses, orders & products, etc.)
- Junction (Bridge/Join) Tables
  - An intermediate table to help store many-to-many relationship



Hands-on Part 1: Warm up

- Retrieve data: `SELECT...FROM...`
- Sort retrieved data: `SELECT...FROM...ORDER BY...`
- Filter data: `SELECT...FROM...WHERE...`
  - `IN`, `NOT`, `LIKE` and its wildcard: `%`, `_`, GLOB its wildcard: `*`, `?`, and more.
- Create calculated fields
  - mathematical calculations (e.g. `+`, `-`, `*`, `/`)
  - data manipulation functions (e.g. `DATE()`, `||`)

SQL Syntax Order vs Execution Order (1)

- Refer to column alias in `WHERE` clause

```
SELECT column_name/col_expression AS alias_name
FROM table
WHERE alias_name...
```

  - Works in SQLite, but…*

- SQL syntax order
  - `SELECT… FROM… WHERE… ORDER BY… LIMIT…`

- SQL execution order
  - `FROM, WHERE, SELECT, ORDER BY, LIMIT`

* Test it with T-SQL for MS SQL Server at SQL Tryit from w3schools.

Hands-on Part 2: Summarize and Group Data

- Summarize data using aggregate functions (e.g. `COUNT()`, `MIN()`, `MAX()`, and `AVG()`).
- Group data and filter groups: `SELECT...FROM...GROUP BY...HAVING...`
- SELECT clause ordering: `SELECT...FROM...WHERE...GROUP BY...HAVING...ORDER BY...`
- Filter data by subquery: `SELECT...FROM...WHERE...(SELECT...FROM...)`

SQL Syntax Order vs Execution Order (2)

- Refer to column alias in HAVING clause

```
SELECT col1, agg_fun(col2) AS alias_name
FROM table
GROUP BY col1
HAVING alias_name...
```

  - Works in SQLite, but…

- SQL syntax order
  - `SELECT… FROM… WHERE… GROUP BY… HAVING… ORDER BY… LIMIT…`

- SQL execution order
  - `FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT`

* Test it with T-SQL for MS SQL Server at SQL Tryit from w3schools.

“Bare Columns” Works in SQLite, But…

- “Bare columns”: columns in a SELECT statement that are neither part of the GROUP BY clause nor wrapped in an aggregate function

```
SELECT FirstName, MIN(BirthDate)
FROM Employees;
```

```
SELECT Country, FirstName, MIN(BirthDate)
FROM Employees
GROUP BY Country;
```

- SQLite allows “bare columns”
  - the query result depends on aggregate function used
  - many other DB engines don’t allow it (e.g., PostgreSQL or MS SQL Server)

* Test it with T-SQL for MS SQL Server at SQL Tryit from w3schools.

Hands-on Part 3: Join Tables

- Inner join: `SELECT...FROM...INNER JOIN...ON...`
- Left join: `SELECT...FROM...LEFT JOIN...ON...`
- Other join variations.

Join – Inner Join

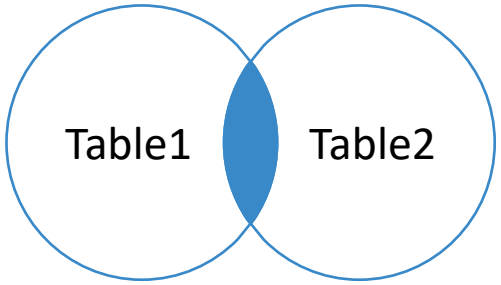


Table1

pk	t1c1
1	a
2	b

Table2

fk	t2c1
1	c
1	d
3	e

```
SELECT *
FROM Table1
  INNER JOIN Table2
  ON Table1.pk = Table2.fk;
```

pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d

Join – Left (Outer) Join

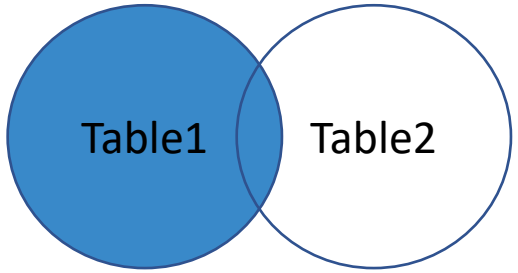


Table1

pk	t1c1
1	a
2	b

Table2

fk	t2c1
1	c
1	d
3	e

```
SELECT *
FROM Table1
  LEFT JOIN Table2
  ON Table1.pk = Table2.fk;
```

pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
2	b	null	null

Join - Left (Outer) Join With Exclusion

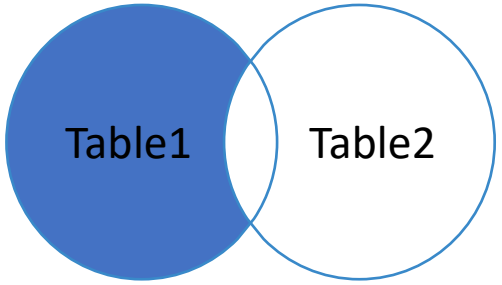


Table1

pk	t1c1
1	a
2	b

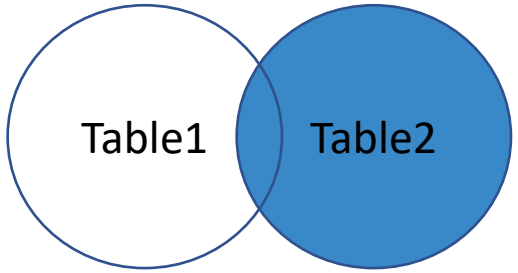
Table2

fk	t2c1
1	c
1	d
3	e

```
SELECT *
FROM Table1
  LEFT JOIN Table2
  ON Table1.pk = Table2.fk
WHERE Table2.fk is NULL;
```

pk	t1c1	fk	t2c1
2	b	null	null

Join – Right Outer Join*




```
SELECT *
FROM Table2
  LEFT JOIN Table1
  ON Table2.fk = Table1.pk
---------------------------
SELECT *
FROM Table1
  RIGHT JOIN Table2
  ON Table1.pk = Table2.fk;
```

Table1

pk	t1c1
1	a
2	b

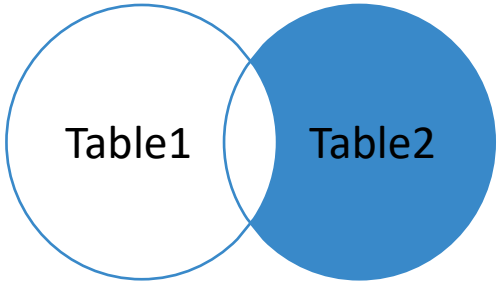
Table2

fk	t2c1
1	c
1	d
3	e

pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
null	null	3	e

Note: SQLite supports the RIGHT JOIN clause starting with version 3.39.0.

Join - Right Outer Join With Exclusion*




```
SELECT *
FROM Table2
  LEFT JOIN Table1
  ON Table2.fk = Table1.pk
WHERE Table1.pk is NULL;
---------------------------
SELECT *
FROM Table1
  RIGHT JOIN Table2
  ON Table1.pk = Table2.fk
WHERE Table1.pk is NULL;
```

Table1

pk	t1c1
1	a
2	b

Table2

fk	t2c1
1	c
1	d
3	e

pk	t1c1	fk	t2c1
null	null	3	e

Join – Full Outer Join

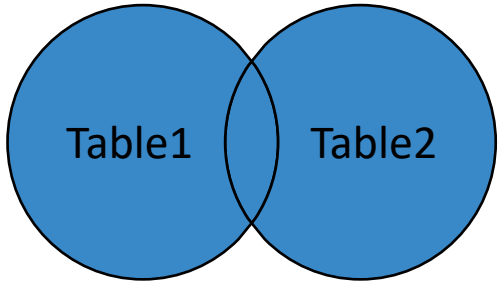


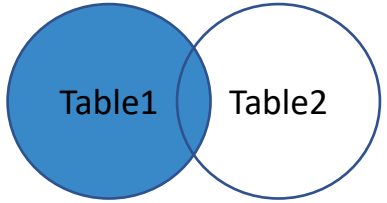
Table1

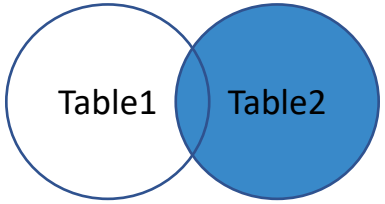
pk	t1c1
1	a
2	b

Table2

fk	t2c1
1	c
1	d
3	e

```
SELECT pk, t1c1, fk, t2c1
FROM Table1
  LEFT JOIN Table2
  ON Table1.pk = Table2.fk
UNION
SELECT pk, t1c1, fk, t2c1
FROM Table2
  LEFT JOIN Table1
  ON Table2.fk = Table1.pk;
```





pk	t1c1	fk	t2c1
1	a	1	C
1	a	1	D
2	b	null	null
null	null	3	e

```
SELECT pk, t1c1, fk, t2c1
FROM Table1
  FULL OUTER JOIN Table2
  ON Table1.pk = Table2.fk
```

Note: SQLite supports the FULL OUTER JOIN clause starting with version 3.39.0.

Join – Full Outer Join With Exclusion*

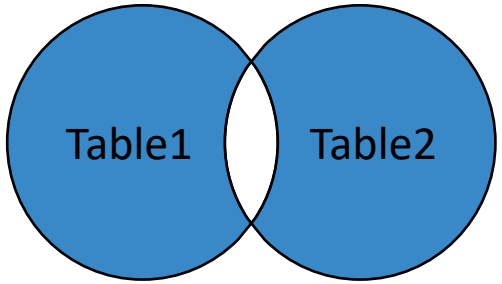


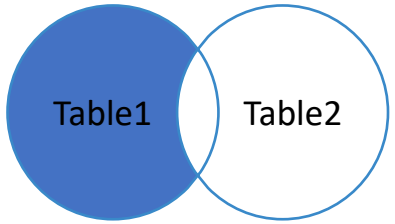
Table1

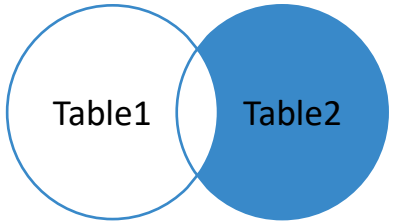
pk	t1c1
1	a
2	b

Table2

fk	t2c1
1	c
1	d
3	e

```
SELECT pk, t1c1, fk, t2c1
FROM Table1
  LEFT JOIN Table2
  ON Table1.pk = Table2.fk
WHERE Table2.fk is NULL
UNION
SELECT pk, t1c1, fk, t2c1
FROM Table2
  LEFT JOIN Table1
  ON Table2.fk = Table1.pk
WHERE Table1.pk is NULL;
```





pk	t1c1	fk	t2c1
2	b	null	null
null	null	3	e

```
SELECT pk, t1c1, fk, t2c1
FROM Table1
  FULL OUTER JOIN Table2
  ON Table1.pk = Table2.fk
WHERE Table2.fk is NULL
  OR Table1.pk is NULL
```

SQL Syntax Order vs Execution Order (3)

- SQL is a *declarative* language, i.e., not a *procedural* language
  - You tell the system what you want to compute, not how to compute it
  - Behind the scenes, a query planner figures out the best way (a physical execution order) to compute it

- SQL syntax order
  - `SELECT (DISTINCT)… FROM… [INNER/LEFT] JOIN… ON… WHERE… GROUP BY… HAVING… ORDER BY… LIMIT…`

- SQL logical execution order
  - `FROM, JOIN…ON…, WHERE, GROUP BY, HAVING, SELECT (DISTINCT), ORDER BY, LIMIT`

Logical Execution Order Summary

Logical Order	Clause	What it does
1	FROM / JOIN	Identifies and combines the target tables to establish the baseline dataset.
2	WHERE	Filters individual rows before any groupings happen.
3	GROUP BY	Collapses rows into grouped subsets for aggregation.
4	HAVING	Filters those grouped subsets using aggregate functions.
5	SELECT	Computes the output columns, expressions, and window functions.
6	DISTINCT	Eliminates duplicate rows from the computed select list.
7	ORDER BY	Sorts the final output rows.
8	LIMIT / OFFSET	Restricts the final output to a specific number of rows.

Hands-on Part 4: Others

- CTE and temporary table
- Self-join (and a glance of window functions)
- CASE keyword
- UNION keyword

UNION vs UNION All

- Syntax: Query 1 UNION (ALL) Query 2

- Both combine rows
- UNION removes duplicated rows, but UNION All doesn't

- Rules
  - # of cols in both queries must be the same
  - Corresponding cols must have compatible data types
  - Col names of the first query determine the col names of the combined result

Ref. https://www.sqlitetutorial.net/sqlite-union/

UNION vs UNION All – An Example (Part 1)

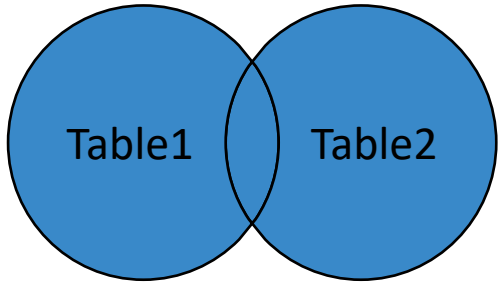


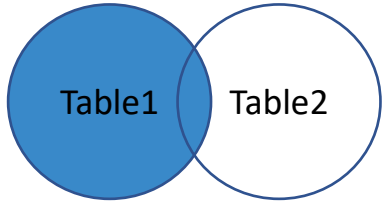
Table1

pk	t1c1
1	a
2	b

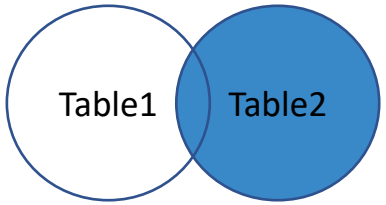
Table2

fk	t2c1
1	c
1	d
3	e

```
SELECT pk, t1c1, fk, t2c1
FROM Table1
  LEFT JOIN Table2
  ON Table1.pk = Table2.fk
UNION
SELECT pk, t1c1, fk, t2c1
FROM Table2
  LEFT JOIN Table1
  ON Table2.fk = Table1.pk;
```



pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
2	b	null	null



pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
null	null	3	e

pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
2	b	null	null
null	null	3	e

UNION vs UNION All – An Example (Part 2)

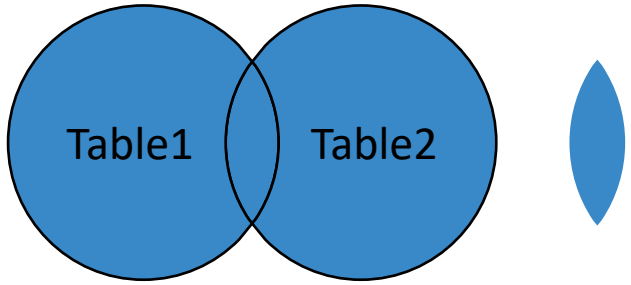


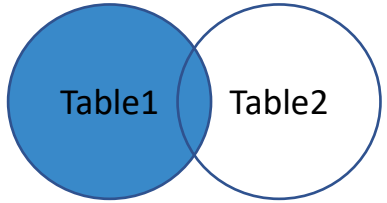
Table1

pk	t1c1
1	a
2	b

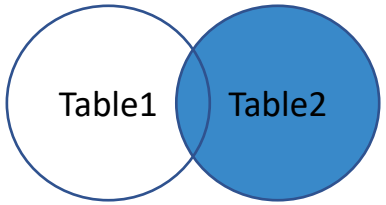
Table2

fk	t2c1
1	c
1	d
3	e

```
SELECT pk, t1c1, fk, t2c1
FROM Table1
  LEFT JOIN Table2
  ON Table1.pk = Table2.fk
UNION ALL
SELECT pk, t1c1, fk, t2c1
FROM Table2
  LEFT JOIN Table1
  ON Table2.fk = Table1.pk;
```



pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
2	b	null	null



pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
null	null	3	e

pk	t1c1	fk	t2c1
1	a	1	c
1	a	1	d
2	b	null	null
1	a	1	c
1	a	1	d
null	null	3	e

Many things we didn't cover

- Insert data (`INSERT INTO…VALUES…; INSERT INTO…SELECT…FROM…`)
- Update data (`UPDATE…SET…WHERE…`)
- Delete data (`DELETE FROM…WHERE…`)
- Manipulate tables (`CREATE TABLE…; ALTER TABLE…; DROP TABLE…`)
- Views (`CREATE VIEW…AS…`)

Ref. SQLite Tutorial - An Easy Way to Master SQLite Fast

The list goes on and on

- Stored procedures (not supported in SQLite)
- User-defined Functions (not supported in SQLite)
- Transaction processing
- Cursors (going through table row by row)
- WINDOW functions (SQLite >= version 3.25; only had a glance of it)
- Query optimization
- DB permissions & security
- …

Ref. A stack overflow discussion on What is “Advanced” SQL.